Feedback Report Homework Assignment 2

Mursal Stella Mohammadi

October 22, 2026

Correction Feedback integrated

The feedback was:

1. use MPI_Wtime instead of UNIX clocktime gathering
2. uses scatterv for data exchange, including halo rows. Scatterv does not allow for re-reading read memory regions. No circumvention or discussion regarding this.
3. optimised version is borderline, but will allow once scaling is done sufficiently, to show whether it actually pays off.
4. need to see better isolation of OpenMP and MPI experiments - you should scale both of them to a certain degree.
5. strong and weak scaling: number of processes too low. 60+ at least. need to see more network communication and overhead effects.

1. Feedback point

Time is now measured with MPI_Wtime() in the parallel versions. Then the max is choosen over all ranks.

2. Feedback point

I have now fixed this with using MPI_Sendrecv() to exchange the halo and used them in the function ExchangeHalo(). For this i needed to adapt the Scatter table (updated BuildScatterGather function) so it scatters the image according to the rows without the halo. So futher I used a temporary variable true_tile in which i scatter the image. And then copy it into tile_img according the existance of an upper halo or not in both the parallel and optimized version.

```
static void ExchangeHalo(int rank, RGB *tile_img, const int width, const int upper_halo,
const int lower_halo, const int rows, MPI_Datatype rowtype)
{
    RGB *own = tile_img + (size_t)upper_halo * width; //move pointer according to upper_halo
    int upper_neighbour;
    if (upper_halo){
        upper_neighbour = rank - 1;
    }else{
        upper_neighbour = MPI_PROC_NULL;
    }
    int lower_neighbour;
    if (lower_halo){
    lower_neighbour = rank + 1;
    } else {
        lower_neighbour = MPI_PROC_NULL;
    }
    //lower halo send + rec; send first row as lower halo to upper rank neighbour.
    MPI_Sendrecv(own, 1, rowtype, upper_neighbour, 0,
        //and receive first row from lower naighbour rank as lower halo
        own + (size_t)rows * width, 1, rowtype, lower_neighbour, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    //upper halo send + rec; send last row to lower nieghbour rank as upper halo
    MPI_Sendrecv(own + (size_t)(rows - 1) * width, 1, rowtype, lower_neighbour, 1,
    //receive last row from upper neighbour as upper halo
    tile_img, 1, rowtype, upper_neighbour,
    1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
```

and inside the main

```
...
    if (rank == 0){
        true_rows = (int *)malloc(size * sizeof(int));
        true_start = (int *)malloc(size * sizeof(int));
        BuildScatterGather(size, height, true_rows, true_start);
        // Fill the arrays for scatter and gather tables
    }
    // Init mem for tile with halo rows, and tile output without halo rows
    RGB *true_tile = (RGB *)malloc((size_t)rows * width * sizeof(RGB));
    RGB *tile_img = (RGB *)malloc((size_t)used_rows * width * sizeof(RGB));
...
    // rank 0 sends tiles and
    each rank receives its tile to work upon
    MPI_Scatterv(img, true_rows, true_start,
    rowtype, true_tile, rows, rowtype, 0, MPI_COMM_WORLD);
    //shift tile accordingly into tile_img
    memcpy(tile_img + (size_t)upper_halo * width,
    true_tile, (size_t)rows * width * sizeof(RGB));
    free(true_tile);
...
    ExchangeHalo(rank, tile_img, width, upper_halo, lower_halo, rows, rowtype);
...
```

3. Feedback point

I revised the optimized version and extended it with a optimization of the MPI_Type. The result tiles are now gathered as single grayscale (unsigned char, 1 byte per pixel) instead of RGB (3 bytes per pixel), reducing the MPI_Gatherv data by a factor of three. To expand from Grayscale to RGB i have used a helper function GrayToRGB().

```
static void GrayToRGB(const unsigned char *grayscale,
RGB *img, size_t img_size) {
    #pragma omp parallel for schedule(static)
    for (size_t i = 0; i < img_size; i++) {
        img[i].red = grayscale[i];
        img[i].green = grayscale[i];
        img[i].blue = grayscale[i];
    }
}
```

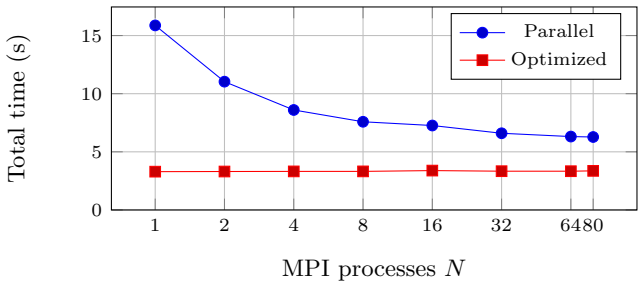
4. Feedback point

I have changed the time measuring point. Now I measure the Scattering time, computing of filtering the gathering time and total time elapsed.

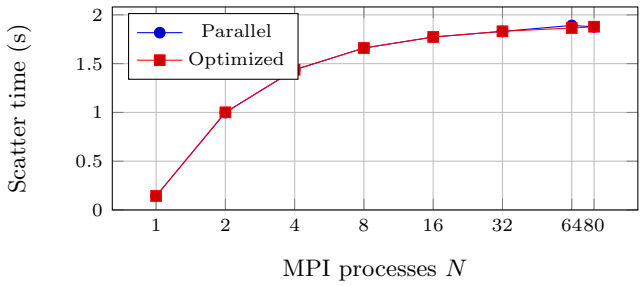
Scatter: s Compute: s Gather: s Total: s (Scatter + Compute + Gather)

All runs use the complete hostfile (all available nodes), one rank per node, $T = 1$. Every message therefore crosses the network. With RPN = 1

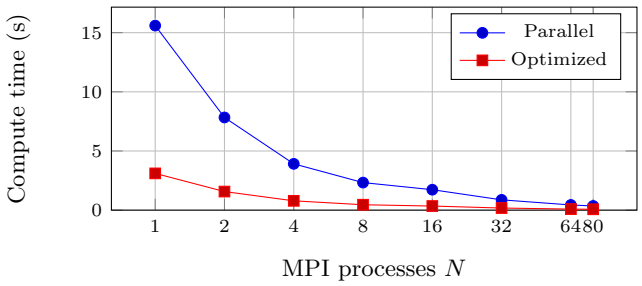
Pure MPI Scaling: T = 1



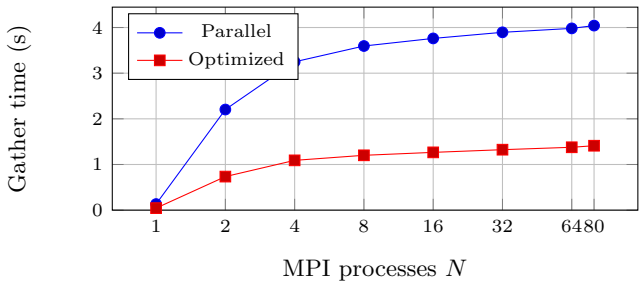
(a) Total time.



(b) Scatter time.



(c) Compute time.



(d) Gather time.

Figure 1: Pure MPI scaling at $T = 1$, broken down by phase: total (top left), scatter (top right), compute (bottom left), and gather (bottom right). All panels share the same x-axis (MPI rank count N).

The compute time of both versions decreases with N, and the optimized version is faster at every N (3.11 s vs. 15.60 s at $N = 1$, 0.073 s vs. 0.354 s at $N = 80$). The scatter time is identical for both versions (0.14 s at $N = 1$, 1.88 s at $N = 80$), since the same data is distributed. The gather time grows with N in both versions, but is lower in the optimized version because only one byte per pixel is transferred instead of three. With rising N, scatter and gather dominate. The total time of the optimized version therefore stays constant at about 3.3 s and does not scale with N. It is nevertheless faster than the parallel version at every N.

	Parallel ($T = 1$)			**Optimized** ($T = 1$)		
N	T_{tot} (s)	$S(N)$	$E(N)$	T_{tot} (s)	$S(N)$	$E(N)$
1	15.872	1.000	100%	3.295	1.000	100%
2	11.035	1.438	71.9%	3.307	0.996	49.8%
4	8.600	1.846	46.1%	3.317	0.993	24.8%
8	7.585	2.093	26.2%	3.318	0.993	12.4%
16	7.263	2.185	13.7%	3.387	0.973	6.1%
32	6.596	2.406	7.5%	3.334	0.988	3.1%
64	6.313	2.514	3.9%	3.333	0.989	1.5%
80	6.275	2.530	3.2%	3.364	0.980	1.2%

Figure 2: Total time, speedup, and efficiency.

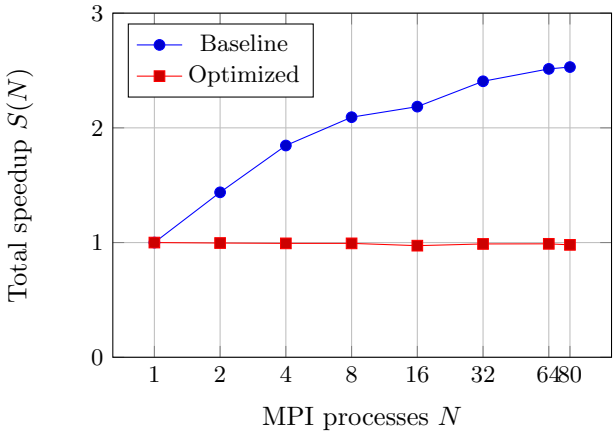
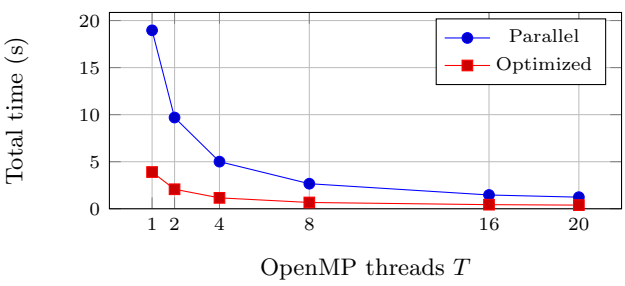


Figure 3: MPI total speedup at $T = 1$.

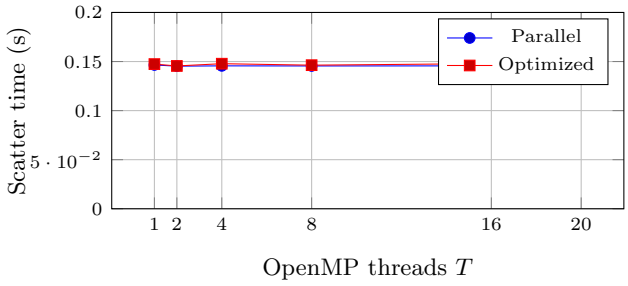
The parallel version achieves a total speedup of 2.53 at $N = 80$. The optimized version shows no speedup over N (0.98 at $N = 80$), because its computation is about 5× faster. In summary, the optimized version is faster than the parallel version for every N.

Pure OpenMP Scaling: N = 1

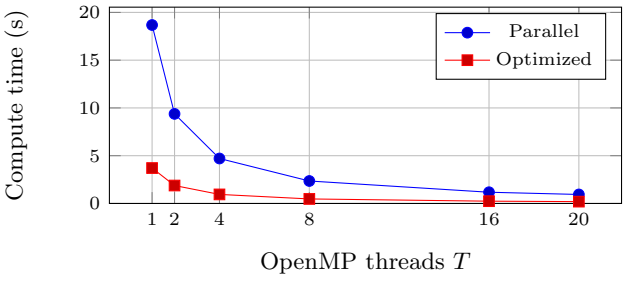
I have conducted the test on only one node, c10-0 slots=20, the same hardware.



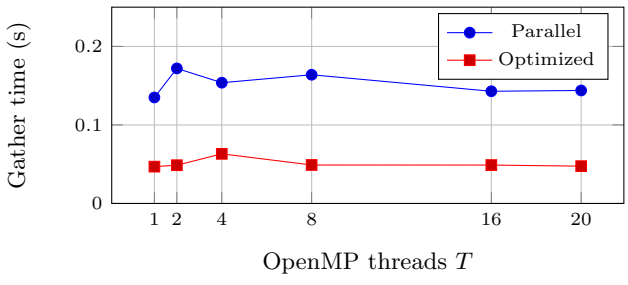
(a) Total time.



(b) Scatter time.



(c) Compute time.



(d) Gather time.

Figure 4: Pure OpenMP scaling at $N = 1$: total (top left), scatter (top right), compute (bottom left), and gather (bottom right)

The compute time of both versions decreases almost proportionally to T (compute efficiency $\geq 99\,\%$ for the parallel and $\geq 96.8\,\%$ for the optimized version at $T = 20$), and the optimized version is faster at every T (3.71 s vs. 18.68 s at $T = 1$, 0.19 s vs. 0.94 s at $T = 20$). The scatter and gather times do not depend on T, since OpenMP does not parallelize them.

	Parallel ($N = 1$)			**Optimized** ($N = 1$)		
T	T_{tot} (s)	$S(T)$	$E(T)$	T_{tot} (s)	$S(T)$	$E(T)$
1	18.962	1.000	100%	3.901	1.000	100%
2	9.695	1.956	97.8%	2.072	1.883	94.2%
4	5.006	3.788	94.7%	1.158	3.370	84.2%
8	2.661	7.126	89.1%	0.668	5.838	73.0%
16	1.464	12.956	81.0%	0.435	8.968	56.1%
20	1.231	15.402	77.0%	0.387	10.074	50.4%

Figure 5: Total time, speedup, and efficiency.

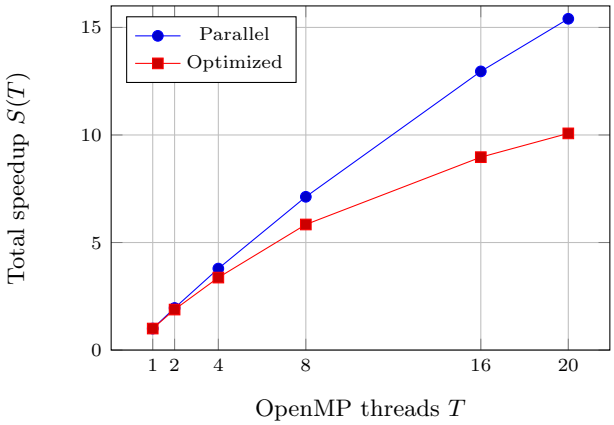


Figure 6: OpenMP total speedup at $N = 1$.

The parallel version achieves a total speedup of 15.4 at $T = 20$ (efficiency 77 %), the optimized version 10.1 (efficiency 50 %).

5. Feedback point

Strong scaling

I have mostly used the ./filter.sh script to use the 25 nodes with 8 slots as a base for testing the parallel and optimized version. In both the strong and weak scaling. To not oversubscribe the amount of Threads T will be kept constant at T = 1. But the new variable Ranks Per Node (RPN) will be choosen to alter the amounts of processes running on one node.

As I measured N = 1 as baseline for parallel and optimised version T(1) is accordingly:

- Parallel Baseline: T(1) = 9.952930602 s
- Optimised Baseline: T(1) = 2.035640494 s

Communication share is calculated as the follwing: (Scatter + Gather) / Total × 100 % and for strong scaling large.bmp with 210 MB is used.
All runs use the 25 nodes with 8 cores each (`hostfile_8`), RPN = 8, $T = 1$.

Table 1: Strong Scaling – Parallel (Baseline $T = 1$)

N	Scatter (s)	Compute (s)	Gather (s)	Total (s)	Comm. %	Speedup $S(N)$	Efficiency $E(N)$
1	0.047922	9.859217	0.045790	9.952931	0.94	1.000	1.000
8	0.043069	1.300010	0.049128	1.392224	6.62	7.149	0.894
16	0.970326	0.670025	1.927237	3.576451	81.01	2.783	0.174
32	1.439260	0.338845	2.882409	4.660714	92.73	2.136	0.067
64	1.683197	0.174661	3.387924	5.247708	96.66	1.897	0.030
128	1.837233	0.091005	3.687278	5.614926	98.38	1.773	0.014

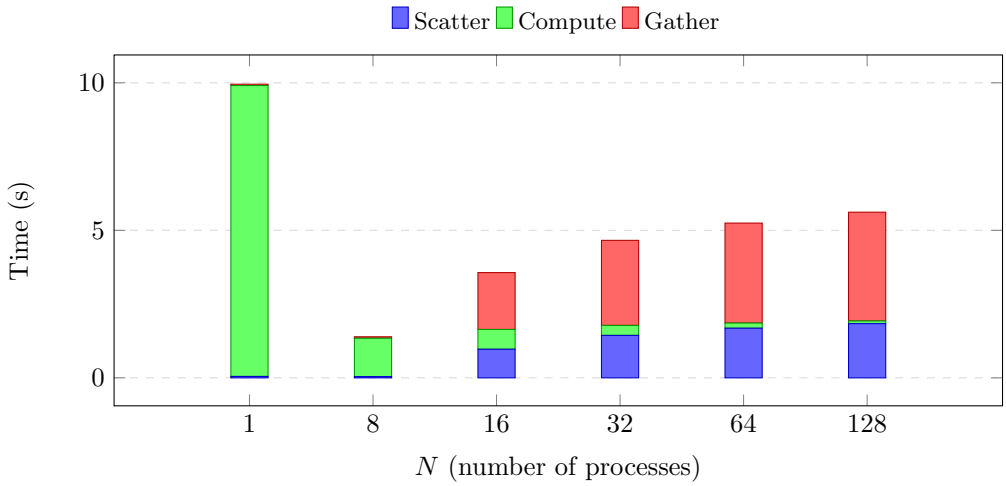


Figure 7: Runtime for strong scaling (Parallel $T = 1$).

Up to $N = 8$ all ranks run on a single node (RPN = 8) and scatter and gather are memory copies. the communication share is only 6.6 % and the efficiency is 89 %. At $N = 16$ the ranks are distributed over two nodes and the data of the remote ranks has to be transferred over the network for the first time. The scatter time rises from 0.04 s to 0.97 s and the gather time from 0.05 s to 1.93 s, while the compute time only halves (1.30 s to 0.67 s). As a result, the total time increases from 1.39 s to 3.58 s, the communication share jumps to 81 %, and the efficiency drops from 89 % to 17 %. The optimised version behaves the same way. The drop is therefore not caused by the computation, but by the transition from shared memory to the network.

Table 2: Strong Scaling – Optimised (Baseline $T = 1$)

N	Scatter (s)	Compute (s)	Gather (s)	Total (s)	Comm. %	Speedup $S(N)$	Efficiency $E(N)$
1	0.048974	1.970479	0.016187	2.035640	3.20	1.000	1.000
8	0.043365	0.264846	0.020344	0.328427	19.40	6.198	0.775
16	0.977801	0.133945	0.650088	1.762599	92.38	1.155	0.072
32	1.438411	0.070264	0.977728	2.486605	97.17	0.819	0.026
64	1.693009	0.035652	1.165763	2.890267	99.00	0.704	0.011
128	1.841588	0.019852	1.323458	3.180258	99.51	0.640	0.005

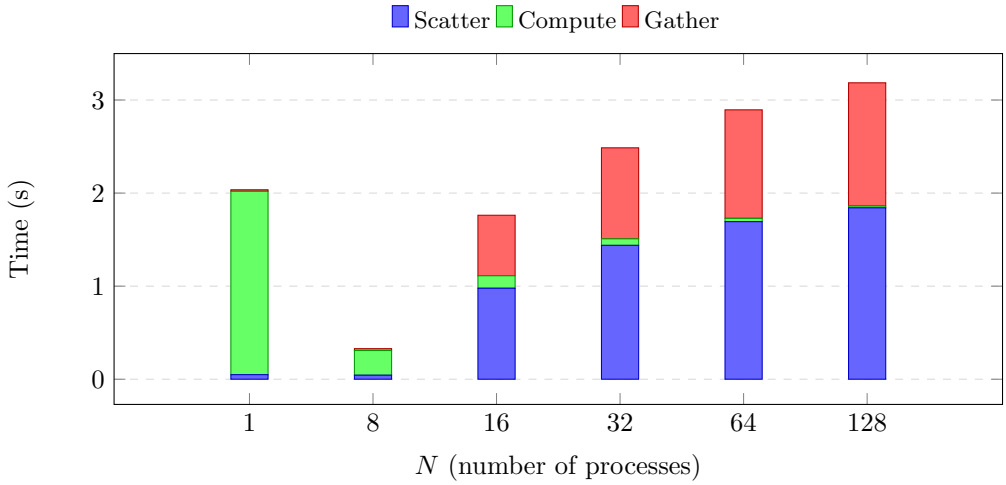


Figure 8: Runtime breakdown for strong scaling (Optimised $T = 1$).

The optimisation reduces Compute and Gather a lot (Gather for exmple. 3.69 → 1.32 s at N=128), but Scatter stays basically constant (1.84 s) and dominates at N=16. So efficiency stays low as an result.

Network communication

Table 3: Fixed-N measurements for PARALLEL and OPTIMISED variants (RPN = ranks per node; times in seconds; Comm. in %).

N	RPN	Scatter	Compute	Gather	Total	Comm. %
PARALLEL						
64	8	1.695 635	0.172 721	3.385 389	5.246 480	96.90
64	4	1.806 132	0.164 808	3.629 200	5.597 252	97.11
64	3	1.840 485	0.163 454	3.706 687	5.713 425	97.06
128	8	1.838 031	0.175 403	3.696 373	5.707 669	96.93
128	4	1.900 713	0.176 461	3.833 807	5.908 036	97.07
80	1	1.860 635	0.352 226	4.047 255	6.262 630	94.37
OPTIMISED						
64	8	1.690 298	0.035 928	1.164 636	2.888 892	98.80
64	4	1.803 220	0.036 292	1.250 155	3.087 782	98.86
64	3	1.831 603	0.034 855	1.272 063	3.140 355	98.79
128	8	1.838 344	0.036 525	1.323 570	3.198 532	98.86
128	4	1.904 310	0.037 545	1.365 724	3.305 094	98.92
80	1	1.861 718	0.072 254	1.420 599	3.357 479	97.85

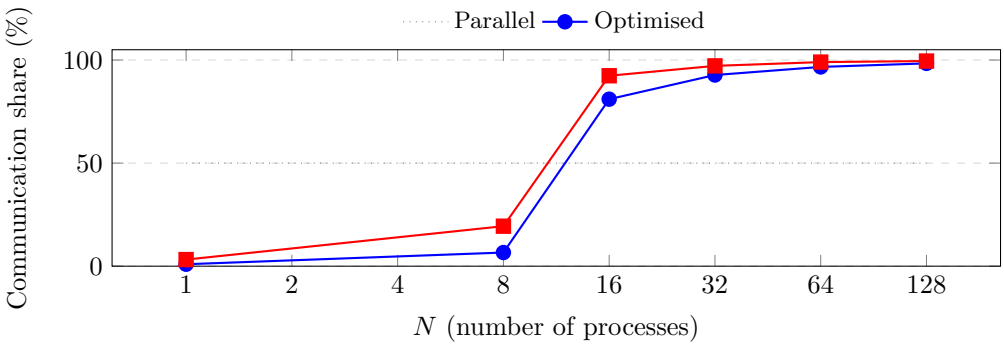


Figure 9: Communication share vs. N – the overhead effect. Small N is compute-bound. beyond $N = 16$ both variants become communication bound (>80 %), which explains the collapsing speedup.

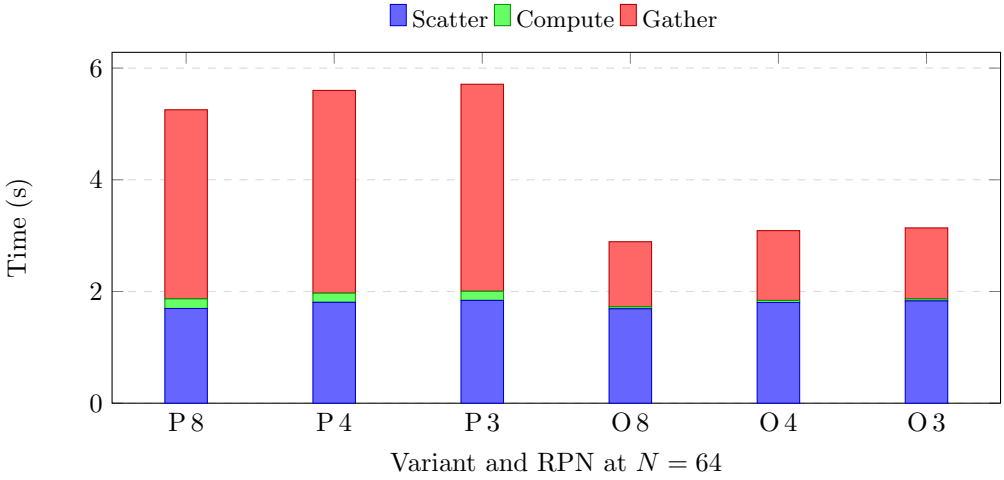


Figure 10: Runtime breakdown vs. RPN at $N = 64$ (P = Parallel, O = Optimised).

RPN=8 with its shared memory is fastest as the shared memory reduces the gather (less network communication). With N=64 vs. N=128 the doubeling of processes doesnt bring speedup. With RPN=3 vs. 8 (N=64 parallel): 5.71 vs. 5.25 s → around 9 % slower through network comm. With N=80 RPN=1 and no shared memory so accordingly it s the slowest.

Weak scaling

During weak scaling the problem size per process is hold constant (here RPN = 4 is fixiated and the image grows with N). I also start with N = 8 so that I reach 60+ processes with the images I have prepared. Idealy Total time i conatant but with increasing processes Scatter and Gather time also grows.
N = 8: wallpaper.bmp (23,7 MB)
N = 16: wallpaper_2.bmp (47,4 MB)
N = 32: wallpaper_4.bmp (94,9 MB)
N = 64: wallpaper_8.bmp (189 MB)
I have used magick to append the image.

Table 4: Weak Scaling – Parallel (RPN=4)

N	Scatter (s)	Compute (s)	Gather (s)	Total (s)	Comm. %	Efficiency $T(8)/T(N)$
8	0.113477	0.145375	0.224327	0.484172	69.77	1.000
16	0.327172	0.144834	0.656202	1.129062	87.06	0.429
32	0.761045	0.146341	1.532575	2.440103	94.01	0.198
64	1.629191	0.145438	3.324021	5.096195	97.15	0.095

Compute time is nearly perfectly constant. But scatter and gather time grows with N.

Table 5: Weak Scaling – optimized (RPN=4)

N	Scatter (s)	Compute (s)	Gather (s)	Total (s)	Comm. %	Efficiency $T(8)/T(N)$
8	0.112826	0.029963	0.079343	0.222896	86.24	1.000
16	0.328666	0.029800	0.227841	0.587124	94.77	0.380
32	0.763599	0.029621	0.528393	1.321706	97.76	0.169
64	1.630718	0.029640	1.127827	2.785267	99.00	0.080

In weak scaling the work per process is constant, so the ideal run time is constant s i sid previously. This is not the case here. the total time grows from 0.48 s to 5.10 s (parallel) and from 0.22 s to 2.79 s (optimized) when going from $N = 8$ to $N = 64$. The compute time per process stays constant (0.145 s and 0.030 s), but scatter and gather grow roughly in proportion to the image size, because all data has to pass through rank 0. Communication makes up 97 % (parallel) and 99 % (optimized) of the total time at $N = 64$. The optimized version is faster in absolute terms (2.79 s vs. 5.10 s at $N = 64$), but its weak efficiency is lower, because its compute time is smaller and the communication share is larger.